

Maintainer's Manual

Introduction

This maintainer's manual presents the design, implementation, and maintenance of the file-content search feature developed for Zulip. The goal of this feature is to enable efficient information retrieval within Zulip by allowing users to search and filter content directly inside uploaded documents. This reduces time spent manually searching through message history.

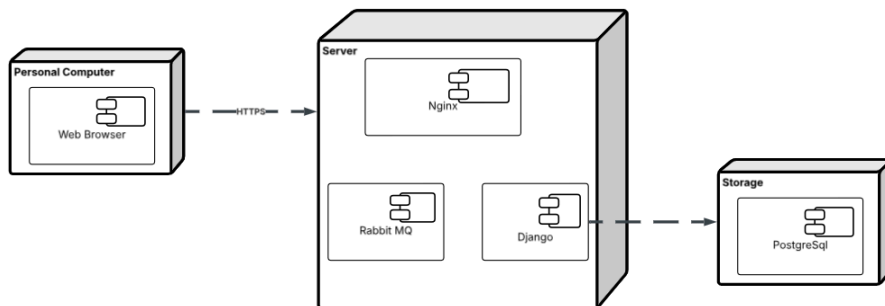
System Design

Architectural Overview

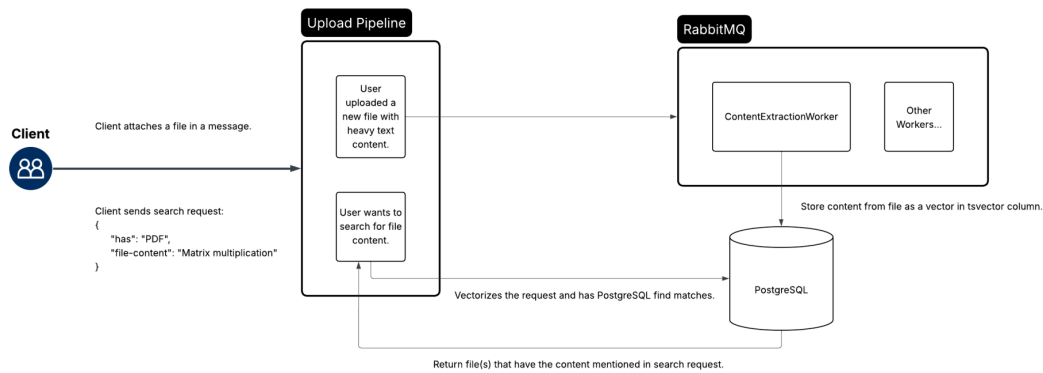
The feature integrates with Zulip's existing full-stack architecture, utilizing a distributed pipeline for text extraction.

- Client/Web Browser: Users interact with the search bar using the `file-content` tag.
- Server (Django/Nginx): Handles incoming search requests and file uploads.
- Upload Pipeline (RabbitMQ): When a user uploads a file, the system enqueues a background job via RabbitMQ.
- ContentExtractionWorker: A background worker that extracts text from various file formats (PDF, DOCX, TXT, PNG, JPG).
- Storage (PostgreSQL): Extracted text is stored as a vector in a dedicated column within the `Attachment` table. Search queries are vectorized to find matches using PostgreSQL's full-text search capabilities.

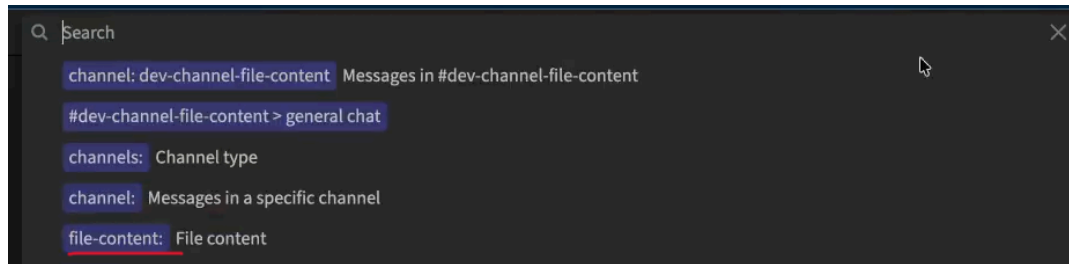
Deployment Diagram



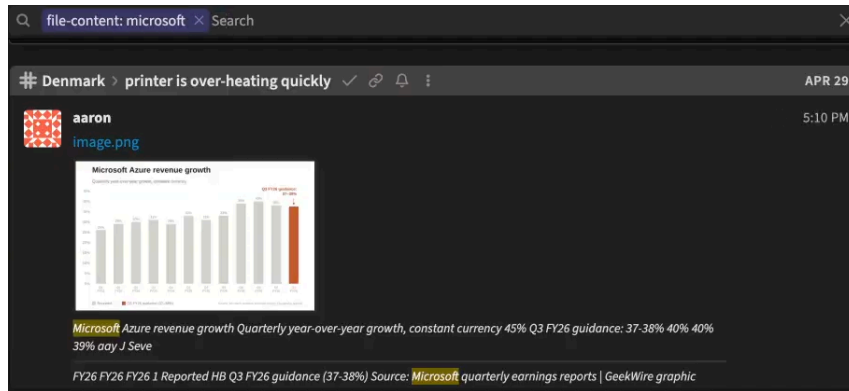
Component Diagram



UI Design



The file-content search feature is presented as a 'file-content' search tag in the Zulip search bar.



Relevant file references are displayed with excerpts and highlighted text for specific matches in the file.

Database Schema

The feature introduces a new `AttachmentContent` model in `zerver/models/messages.py` to store extracted text and full-text search data for uploaded attachments. It is kept separate from the existing `Attachment` model so that the core attachment model stays clean and so that partial or failed extractions can be handled gracefully. An `Attachment` with no corresponding `AttachmentContent` row simply has not been processed yet.

The `AttachmentContent` model contains the following fields:

- **attachment (OneToOneField)**: links to the corresponding Attachment record via a one-to-one relationship, deleted automatically when the parent Attachment is deleted.
- **extraction_status (PositiveSmallIntegerField)**: tracks extraction state, one of PENDING (1), SUCCESS (2), FAILED (3), or UNSUPPORTED (4).
- **extracted_text (TextField)**: stores the raw text extracted from the file, null until extraction succeeds, used for regex search.
- **search_tsvector (SearchVectorField)**: stores the PostgreSQL tsvector derived from extracted_text, used for full-text keyword search queries.
- **last_attempted (DateTimeField)**: records the timestamp of the most recent extraction attempt.

A GIN index is built on search_tsvector with fastupdate=False, following the same pattern as Message.search_tsvector. This allows PostgreSQL to look up matching attachments instantly without scanning every row in the table. The corresponding migration is 0779_attachmentcontent.py.

Requirements Analysis and Specification

Functional Requirements

1. The system shall have a `file-content` tag in the search bar.
2. The system shall enqueue a background worker upon file upload to extract text from uploaded pdf, docx, txt, png, and jpg files.
3. The system shall display excerpts of extracted text from relevant search results with the search words highlighted.
4. The system shall support multi-word file-content search.
5. The system shall support substring-based file-content search.

Non-Functional Requirements

1. The system shall fetch and display relevant file-content search results in under 5 second.
2. The file content extraction shall be asynchronous and not create delays or glitches for subsequent uploads, messages, and searches.

Test Facilities

This section documents how the file-content search feature is tested, how a future maintainer can run and reproduce those tests to guard against regressions, how to generate coverage data, and how to add new tests.

Existing Tests

Our feature is covered by a combination of unit tests and integration tests, with manual testing used for UI changes that are not practical to assert on programmatically. Unit tests exercise individual functions and components in isolation (for example, the text-extraction helpers and the frontend parsing logic); integration tests exercise whole pipelines end to end (for example, uploading a file, running it through the extraction worker, and retrieving the message through a file-content narrow).

All backend tests are written as standard Zulip test cases: each test class inherits from `ZulipTestCase` (and, where the upload pipeline is involved, also mixes in `UploadSerializeMixin`). Because the tests are ordinary `ZulipTestCase` subclasses, they are automatically discovered by Zulip's test runner and run with the same stock commands as the rest of the Zulip backend suite. The frontend tests are Node-based `.cjs` files that live alongside Zulip's existing frontend tests.

Test files

- **`test_narrow.py` (`FileContentNarrowTest`):** backend integration tests for the file-content narrow operator: matching, stemming, multi-word logic, negation, status-based exclusion, prefix matching, and snippet highlighting.
- **`test_file_content_extraction.py` (`FileContentExtractionWorkerTest`):** text extraction from DOCX and PDF, OCR extraction from images, and a worker sanity check.
- **`test_try_upload.py` (`TestTryUpload`):** verifies an uploaded file's `content_type` is correctly stored on the Attachment record.
- **`test_file_content_performance.py` (`FileContentIndexPerformanceTest`):** opt-in performance tests for the GIN index on `search_tsvector`; skipped by default.
- **`test_upload.py` (`FileQueueTest`):** verifies an upload enqueues a `file_content_extraction` job carrying the attachment id.
- **`test_attachments.py` (`test_attachment_fields_persist`):** verifies the core Attachment fields persist correctly.
- **`web/tests/filter.test.cjs`:** frontend parsing and unparsing of the file-content operator, and that it is only evaluated server-side.
- **`web/tests/search_suggestion.test.cjs`:** frontend typeahead behavior for file-content suggestions and completions.

How to Run Tests

Run the full backend suite:

- `./tools/test-backend`

Run a single backend module, class, or method:

- `./tools/test-backend zerver.tests.test_narrow`
- `./tools/test-backend zerver.tests.test_narrow.FileContentNarrowTest`
- `./tools/test-backend zerver.tests.test_narrow.FileContentNarrowTest.test_stemming`

Run the full frontend suite, or target specific suites by file basename:

- `./tools/test-js-with-node`
- `./tools/test-js-with-node filter search_suggestion`

The performance suite is skipped by default so it does not slow down ordinary test runs. It is gated behind the `RUN_PERF_TESTS` environment variable and must be opted into explicitly:

- `RUN_PERF_TESTS=1 ./tools/test-backend zerver.tests.test_file_content_performance`

The performance tests print timing at each scale and assert that the GIN index produces a meaningful speedup over a sequential-scan baseline. The largest scale generates a substantial number of rows, so this

suite takes noticeably longer than the standard tests; run it when validating index or query-planner changes, not on every commit.

Test Coverage

Both test runners accept a `--coverage` flag that generates the coverage data:

- `./tools/test-backend --coverage`
- `./tools/test-js-with-node --coverage`

The backend `--coverage` run prints a per-file coverage summary to the terminal and writes a full HTML report, whose path is printed at the end of the run. The frontend `--coverage` run produces a coverage report through Zulip's frontend coverage tooling. To check coverage for just the files touched by this feature, read the per-file rows for the relevant modules (for example, `narrow.py`, or `filter` and `search_suggestion` on the frontend).

Coverage results

- Backend: 94%.
- Frontend: 74%.

Adding New Tests

Add new backend tests to the existing test class for the area being changed and follow the helpers already defined there. The narrow tests in particular exercise the real query pipeline rather than asserting against SQL strings, and one detail is easy to miss: the `search_tsvector` column cannot be written through the Django ORM, so it must be populated with raw SQL using the same `zulip.english_us_search` configuration that `by_file_content` uses. Expensive performance or stress tests should be added to `test_file_content_performance.py` and gated with `@skipUnless` on `RUN_PERF_TESTS` so they stay out of the default suite. New frontend tests go in the existing `web/tests/filter.test.cjs` and `web/tests/search_suggestion.test.cjs` files, following the existing `test(...)` blocks. When testing a new operator behavior, cover both the single-word and quoted multi-word operand forms.

Deployment Procedure

The file-content search feature requires no additional deployment steps beyond Zulip's standard development setup. To run the feature, first clone the forked repo. Then, run the command `./tools/provision` to ensure all the newest dependencies are installed. To start the development server, run the command `./tools/run-dev`.